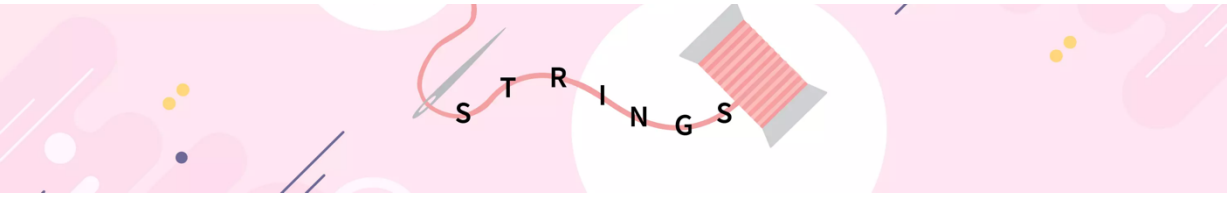
STRINGS

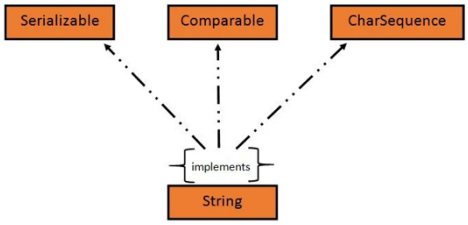


Strings- Introduction

String is an object that represents sequence of characters. In Java, String is represented by String class which is located into java.lang package.

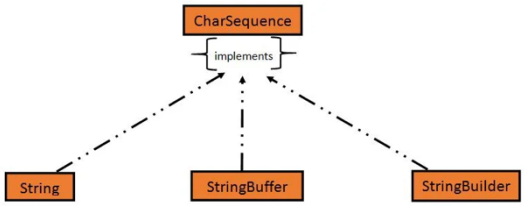
It is probably the most commonly used class in java library. In java, every string that we create is actually an object of type String. One important thing to notice about string object is that string objects are immutable that means once a string object is created it cannot be changed.

The Java String class implements Serializable, Comparable and CharSequence interface that we have represented using the side image.



CharSequnce- Introduction

In Java, CharSequence Interface is used for representing a sequence of characters. CharSequence interface is implemented by String, StringBuffer and StringBuilder classes. This three classes can be used for creating strings in Java.



Declaration and Creation of a String

There are **two** ways to create a String in Java:

1. By string literal- String Literal
2. By new keyword- String Object

And, it's also important that we understand how strings are managed in specialized pool – we will see in Memory Management of Strings.

Memory Management of Strings

When we create a String object using the *new()* operator, it always creates a new object in heap memory. On the other hand, if we create an object using String Literal syntax e.g. “Manu Chaitanya”, it may return an existing object from the **String Pool**, if it already exists. Otherwise, it will create a new String object and put in the string pool for future re-use.

At a high level, both are the String objects, but the main difference comes from the point that *new()* operator always creates a new String object. Also, when we create a String using literal – it is interned.

Strings are **immutable** in nature. Immutable mean ns they have a **constant value**, and even if they are altered, instead of reflecting the alterations in the original string, a new object is created. This immutability is achieved through **String Pool**. Let us look at the concept of String Pool.

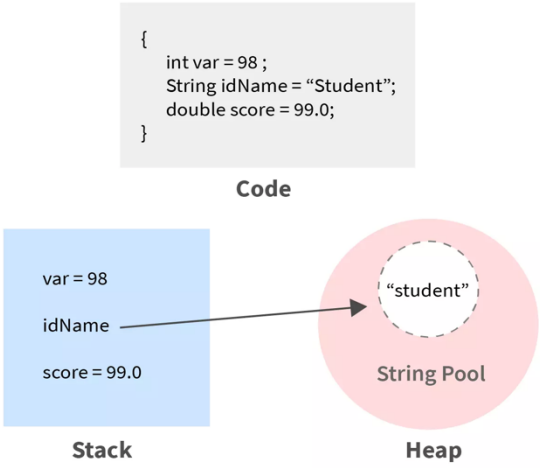
Memory Management of Strings Literal

String Pool in Java is a special storage space in Java Heap memory where string literals are stored. It is also known by the names - **String Constant Pool** or **String Intern Pool**. Whenever a string literal is created, the JVM first checks the String Constant Pool before creating a new String object corresponding to it.

Let us first discuss the memory allocation methods used in Java - **Stack Memory Allocation**, **Heap Memory Allocation and String Pool.**

- In stack memory, only the primitive data types like- int, char, byte, short, boolean, long, float and double are stored.
- Whereas, in the heap memory, non-primitive data types like strings are stored. A reference to this location is held by the stack memory.
- The String Pool is empty by default which is inside the heap memory only, and it is maintained privately by the String class.
- When we create a string literal, the JVM first checks that literal in the String Constant Pool. If the literal is already present in the pool, its reference is stored in the variable.

- However, if the string literal is not found, the JVM creates a new string object in the String Constant Pool and returns its reference.



Let's discuss the need of String Constant Pool:

- When we create a String object, it uses some amount of space in the heap memory.
- Let's say we are creating n number of String objects with the same value, and distinct memory is allocated to each of these string objects (though they all contain the same string).
- This is an inefficient usage of heap memory. In order to escalate the performance of our code and reduce memory usage, JVM optimizes the way in which strings are stored with the help of a string constant pool.

Automatic String Interning

Thanks to the immutability of *Strings* in Java, the JVM can optimize the amount of memory allocated for them by storing only one copy of each literal *String* in the pool. This process is called *interning*.

When we create a String variable and assign a value to it, the JVM searches the pool for a *String* of equal value.

If found, the Java compiler will simply return a reference to its memory address, without allocating additional memory. If not found, it'll be added to the pool (interned) and its reference will be returned.

Memory Management of Strings Object

When we create a String via the new operator, the Java compiler will create a new object and store it in the heap space not the string pool reserved for the JVM.

Every String created like this will point to a different memory region with its own address.

String Literal vs String Object

When we create a *String* object using the *new()* operator, it always creates a new object in heap memory. On the other hand, if we create an object using *String* literal syntax e.g. "Manu Chaitanya", it may return an existing object from the String pool, if it already exists. Otherwise, it will create a new String object and put in the string pool for future re-use.

At a high level, both are the *String* objects, but the main difference comes from the point that *new()* operator always creates a new *String* object. Also, when we create a *String* using literal – it is interned.

Manual Interning

We can manually intern a *String* in the Java String Pool by calling the *intern()* method on the object we want to intern. Manually interning the *String* will store its reference in the pool, and the JVM will return this reference when needed.

Garbage Collection

Before Java 7, the JVM placed the Java String Pool in the PermGen space, which has a fixed size — it can't be expanded at runtime and is not eligible for garbage collection.

The risk of interning Strings in the PermGen (instead of the *Heap*) is that we can get an OutOfMemory error from the JVM if we intern too many Strings.

From Java 7 onwards, the Java String Pool is **stored in the Heap space, which is garbage collected** by the JVM. The advantage of this approach is the **reduced risk of OutOfMemory error** because unreferenced Strings will be removed from the pool, thereby releasing memory.

Immutabilty of String

An immutable object is an **object whose internal state remains constant after it has been entirely created**. In Java, strings are immutable, which means their values cannot be changed after they are created. Once a string object is created, its state (the sequence of characters it represents) cannot be modified.

When we perform methods on a string in Java, such as concatenation or substring extraction, a new string object is created with the modified value, while the original string object remains unchanged. Here we know that there is pass by value in Java so the strings reference is not passed to the methods but the value is copied as parameter in the methods. Thus it doesn't affects the original string instead create a new string object. Although this is not specific to strings but any other data type.

In Java, **the reference to a string can be changed, but the string object's internal state itself remains immutable**. **Let me explain in more detail with an example.**

- The variable `str` holds a reference to a string object with the value "*Hello*". Now, if you assign a new string to the same variable, **the reference will be updated** to point to the new string object.
- After this assignment, the `str` variable references a new string object with the value "*World*". However, the original string object "*Hello*" still exists in memory, and since strings are immutable, its value cannot be changed.
- In this case, the reference to the string `str` has changed**, but the actual string object ("*Hello*" and "*World*") remains immutable.**

Methods of Strings

The **java.lang.String** class provides a lot of built-in methods that are used to manipulate **string in Java**.

1. toUpperCase() and toLowerCase()

The Java String toUpperCase() method converts this String into uppercase letter and String toLowerCase() method into lowercase letter.

Returns: String in uppercase and lowercase letter respectively.

2. trim()

The String class trim() method eliminates white spaces before and after the String.

Returns: String with omitted leading and trailing spaces

3. startsWith() and endsWith()

The method startsWith() checks whether the String starts with the letters passed as arguments and endsWith() method checks whether the String ends with the letters passed as arguments.

Parameter Prefix : Sequence of character Offset: The index from where the matching of the string prefix starts.

Returns: Boolean

4. charAt()

The String class charAt() method returns a character at specified index.

Parameter Prefix : Index

Returns: Character

5. length()

The String class length() method returns length of the specified String.

Returns: Int

6. valueOf()

The String class valueOf() method coverts given type such as int, long, float, double, Boolean, char and char array into String.

Parameter Prefix : The data type value which has to be converted into String.

Returns: String representation of given value

7. concat()

The Java String class concat() method combines specified string at the end of this string. It returns a combined string. It is like appending another string.

Parameter: Another String, i.e., to be combined at the end of this string.

Returns: Combined string

8. contains()

The Java String class contains() method searches the sequence of characters in this string. It returns true if the sequence of char values is found in this string otherwise returns false.

Parameter: Sequence : specifies the sequence of characters to be searched.

Returns: true if the sequence of char value exists, otherwise false.

9. equals()

The Java String class equals() method compares the two given strings based on the content of the string using iteration. If any character is not matched, it returns false. If all characters are matched, it returns true.

The String equals() method overrides the equals() method of the Object class.

Parameter: Another Object, i.e., compared with this string.

Returns: true if characters of both strings are equal otherwise false.

10. equalsIgnoreCase()

The Java String class equalsIgnoreCase() method compares the two given strings on the basis of the content of the string irrespective of the case (lower and upper) of the string. It is just like the equals() method but doesn't check the case sensitivity. If any character is not matched, it returns false, else returns true.

Parameter: str : another string i.e., compared with this string.

Returns: It returns true if characters of both strings are equal, ignoring case otherwise false.

11. indexOf()

The Java String class indexOf() method returns the position of the first occurrence of the specified character or string in a specified string.

There are four overloaded indexOf() method in Java.

No.	Method	Description
1	int indexOf(int ch)	It returns the index position for the given char value
2	int indexOf(int ch, int fromIndex)	It returns the index position for the given char value and from index
3	int indexOf(String substring)	It returns the index position for the given substring
4	int indexOf(String substring, int fromIndex)	It returns the index position for the given substring and from index

Parameters Ch: It is a character value, e.g. 'a'. formIndex: The index position from where the index of the char value or substring is returned. substring: A substring to be searched in this string.

Returns: Index of the searched string or character.

12. isEmpty()

The Java String class isEmpty() method checks if the input string is empty or not. Note that here empty means the number of characters contained in a string is zero.

Returns: true if length is 0 otherwise false.

13. lastIndexOf()

The Java String class lastIndexOf() method returns the last index of the given character value or substring. If it is not found, it returns -1. The index counter starts from zero.

There are four types of lastIndexOf() method in Java.

No.	Method	Description
1	int lastIndexOf(int ch)	It returns last index position for the given char value
2	int lastIndexOf(int ch, int fromIndex)	It returns last index position for the given char value and from index
3	int lastIndexOf(String substring)	It returns last index position for the given substring
4	int lastIndexOf(String substring, int fromIndex)	It returns last index position for the given substring and from index

Parameters Ch: char value i.e. a single character e.g. 'a'. formIndex: index position from where index of the char value or substring is returned. substring: substring to be searched in this string

Returns: Last index of the string

14. replace()

The Java String class replace() method returns a string replacing all the old char or CharSequence to new char or CharSequence. Since JDK 1.5, a new replace() method is introduced that allows us to replace a sequence of char values. The second replace() method is added since JDK 1.5.

There are two types of replace() methods in Java String class.

1. public String replace(char oldChar, char newChar)
2. public String replace(CharSequence target, CharSequence replacement)

Parameters: **oldChar** : old character **newChar** : new character **target** : target sequence of characters

replacement : replacement sequence of characters

Returns: Replaced string

Note: replaceAll(), replaceFirst() are related to replace(), check them once:

https://replit.com/@ManuChaitanya/javaStrings#Main.java

15. substring()

The Java String class substring() method returns a part of the string.

We pass beginIndex and endIndex number position in the Java substring method where beginIndex is inclusive, and endIndex is exclusive. In other words, the beginIndex starts from 0, whereas the endIndex starts from 1.

There are two types of substring methods in Java string.

1. public String substring(int startIndex)
2. public String substring(int startIndex, int endIndex)

Parameters: **startIndex** : starting index is inclusive. **endIndex** : ending index is exclusive

Returns: Specified string

16. toCharArray()

The Java string toCharArray() method converts this string into character array. It returns a newly created character array, its length is similar to this string and its contents are initialized with the characters of this string.

Returns: Character array

17. split()

The java string split() method splits this string against given regular expression and returns a char array.

Parameter: **regex** : regular expression to be applied on string.

limit : limit for the number of strings in array. If it is zero, it will returns all the strings matching regex.

Returns: Array of strings.